

Training density is not evaluation density: a generalization check on DQN and PPO in highway-env

Yousef Sannan

September 2026

1. Problem and motivation

A driving policy is trained on some distribution of traffic and then meets a different one. Rush hour is not a Sunday morning. Standard reinforcement learning results report performance on the same distribution the agent trained on, which says nothing about what happens when that distribution moves.

This report asks one narrow question:

Does an agent trained at a single traffic density generalize to densities it never saw during training, and does the answer differ between DQN and PPO?

The question is worth separating from raw performance because the two can come apart. An algorithm that scores higher on the training distribution is not necessarily the one that degrades more gracefully off it, and if the two disagree then the choice of algorithm for a system that will meet distribution shift is not the same as the choice that wins a benchmark table.

The experiment is deliberately small. One environment, one independent variable, default hyperparameters, three seeds. The point is that the resulting claim is narrow enough to be checked by someone else in an afternoon, not that it is a strong claim. Section 5 says plainly what it does not establish.

2. Experimental setup

2.1 Environment

All runs use highway-fast-v0 from highway-env 1.12.1, a lightweight three-lane highway simulator. The relevant defaults:

Property	Value
Observation	Kinematics, 5 nearest vehicles x 5 features
Action space	DiscreteMetaAction, 5 actions (lane left/right, faster, slower, idle)
Policy frequency	1 Hz
Simulation frequency	5 Hz
Episode length	30 s, terminating early on collision
	Normalized to [0, 1] per step: collision penalty, speed reward

Reward over 20 to 30 m/s, and a small reward for the right-hand lane

Surrounding traffic IDM vehicles

Maximum achievable episode reward is therefore around 30.

2.2 The independent variable

Traffic density is varied through the `vehicles_density` configuration key, which scales the spacing between vehicles.

The obvious knob, `vehicles_count`, turns out not to be the right one. A pilot sweep with a fixed random policy gave bit-identical results for `vehicles_count` of 20, 40 and 80 (mean reward 9.614, crash rate 1.00 in every case). The reason is that highway-env appends extra vehicles further down the road rather than packing them into the same stretch: with 20 vehicles the furthest is about 475 m ahead of the ego vehicle, with 40 it is about 948 m, and with 80 about 1897 m. Inside a 30 second episode the ego vehicle never reaches them, so they cannot affect anything. Only `vehicles_count = 10` differed, and it differed because it removed vehicles from the stretch the ego vehicle actually drives through.

`vehicles_count` is therefore held at its default of 20 throughout, and the three density levels are:

Level	vehicles_density	Note
low	0.5	Half the stock spacing pressure
medium	1.0	The stock environment. Training density.
high	1.5	50% denser than stock

2.3 Training

Both algorithms train at medium density only, using stable-baselines3 2.9.0 with the `MlpPolicy` and **default hyperparameters throughout**. No tuning sweep was run. Each algorithm is trained with three random seeds (0, 1, 2), and both algorithms get an identical budget of **50,000 timesteps** per run. That budget was fixed once, before any results were seen, from the measured throughput of the environment. Six runs in total.

An identical budget matters more than a large one: if DQN and PPO were given different numbers of environment steps, any difference between them would be uninterpretable.

2.4 Evaluation

Every trained agent is evaluated at all three densities, over 100 episodes per density, taking deterministic (greedy) actions.

All agents are evaluated on the same block of environment seeds (10000 to 10099) at a given density. Every agent therefore faces exactly the same traffic at the same density, so differences between agents cannot be an artefact of one of them drawing easier episodes.

Three metrics are recorded per evaluation point:

- **mean episode reward**, the environment's own objective
- **crash rate**, the fraction of episodes ending in a collision
- **mean speed**, in m/s, averaged over all steps of an episode

Results are reported as the mean and standard deviation across the three training seeds.

2.5 Control

A uniform random policy is evaluated through the identical harness, with three seeds of its own. This is the number every trained result is compared against. Running the control through the same code path as the trained agents is deliberate: a bug in how reward, crashes or speed are computed would move the baseline too, and so would not be mistaken for a training effect.

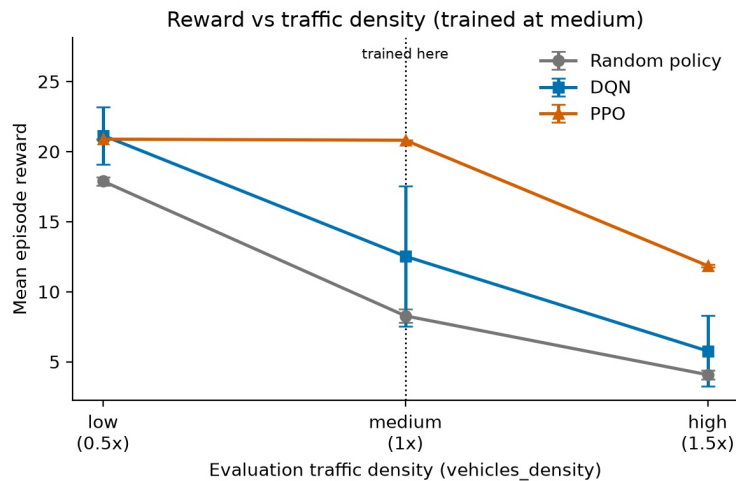
2.6 Reproducibility

Runtime dependencies are pinned to exact versions in `requirements.txt`. All runs were executed on Linux with CPU only. The complete pipeline is three scripts driven from a single configuration module, and the exact commands are in the README.

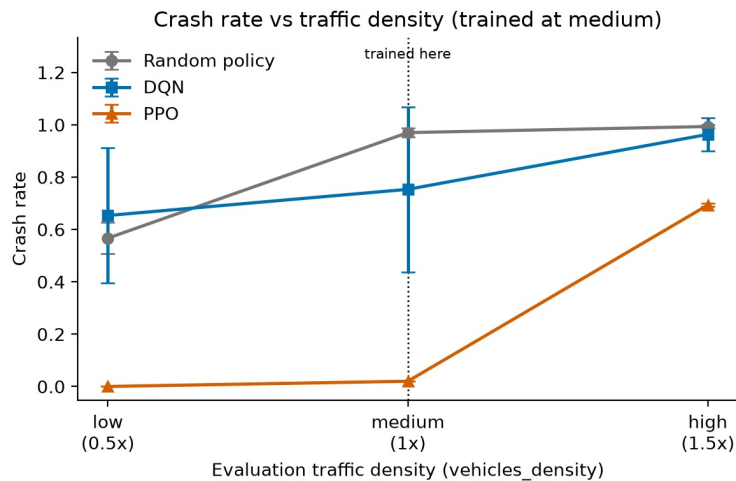
3. Results

All numbers are the mean and standard deviation over the three training seeds, each evaluated over 100 episodes per density. The full per-seed data is in `results/results.csv`.

Agent	Eval density	Mean reward	Crash rate	Mean speed (m/s)	Most common action
Random policy	low (0.5x)	17.89 +/- 0.29	0.57 +/- 0.06	24.62 +/- 0.21	LANE_RIGHT (21% of steps)
Random policy	medium (1x)	8.28 +/- 0.47	0.97 +/- 0.02	23.70 +/- 0.27	FASTER (21% of steps)
Random policy	high (1.5x)	4.08 +/- 0.33	0.99 +/- 0.01	23.11 +/- 0.23	LANE_RIGHT (22% of steps)
DQN	low (0.5x)	21.15 +/- 2.05	0.65 +/- 0.26	28.57 +/- 1.15	SLOWER (75% of steps)
DQN	medium (1x)	12.53 +/- 4.99	0.75 +/- 0.32	25.92 +/- 2.27	LANE_RIGHT (54% of steps)
DQN	high (1.5x)	5.77 +/- 2.53	0.96 +/- 0.06	25.24 +/- 1.91	LANE_RIGHT (54% of steps)
PPO	low (0.5x)	20.88 +/- 0.01	0.00 +/- 0.00	20.03 +/- 0.01	LANE_RIGHT (96% of steps)
PPO	medium (1x)	20.81 +/- 0.01	0.02 +/- 0.00	20.03 +/- 0.02	LANE_RIGHT (95% of steps)
PPO	high (1.5x)	11.85 +/- 0.07	0.69 +/- 0.01	19.57 +/- 0.03	LANE_RIGHT (96% of steps)



Mean episode reward against evaluation traffic density. Error bars are one standard deviation over three training seeds.



Crash rate against evaluation traffic density, same runs.

3.1 The manipulation worked and training did something

Every agent, including the random control, degrades monotonically as density rises, so the independent variable is doing what it is supposed to. Both trained agents beat the random control at every density, so 50,000 timesteps was enough to learn something.

3.2 Read on reward alone, PPO looks like the better generalizer

Relative drop in mean reward going from the training density to high density:

Agent	medium	high	Change
Random policy	8.28	4.08	-51%
DQN	12.53	5.77	-54%
PPO	20.81	11.85	-43%

PPO scores higher at the training density, is essentially flat from medium to low (20.81 to 20.88), and loses less of its score at high density. On reward and crash rate alone, the conclusion would be that

PPO both learns better and transfers better.

3.3 The action distribution says otherwise

PPO takes the same action, LANE_RIGHT, on 95 to 96% of all steps, at every evaluation density. Two of the three seeds take it on 100% of steps, and those two seeds produce **bit-identical** evaluation numbers at all three densities despite having different network weights. Their policies are different functions that happen to have the same argmax everywhere in the observation region visited.

PPO's policy is effectively open loop. It emits the same action whatever the traffic looks like.

That reframes the flat low-to-medium curve entirely. The policy does not adapt to density because the policy does not adapt to anything. It stays flat while the environment tolerates a fixed action and then falls off when it stops: crash rate goes from 0.02 at medium to 0.69 at high, and reward drops 43%.

The mean speed column shows how the score is being earned. PPO holds 20.0 m/s at every density, exactly the bottom of the rewarded speed band of 20 to 30 m/s. The ego vehicle starts every episode at 25 m/s and settles at 20 under this policy. PPO's crash-free record at low and medium density is bought by driving as slowly as the reward function tolerates while collecting the right-lane bonus.

3.4 DQN is reactive but unstable

DQN's action distribution is genuinely mixed (44 to 88% on its most common action, varying with density and seed) and its mean speed is 25 to 29 m/s, so it is responding to observations rather than repeating one action.

It is also unstable at this budget. Per-seed mean reward at the training density is 18.19, 8.78 and 10.61. The spread across seeds (standard deviation 4.99) is larger than the entire gap between DQN's mean and the random baseline (12.53 against 8.28). A single-seed DQN result here could have supported almost any conclusion.

4. Discussion

The direct answer. At a 50,000 timestep budget with default hyperparameters, neither algorithm produced a policy whose competence transfers to unseen densities. Both lose roughly half their score going from the training density to 1.5x, which is the same relative degradation the random policy suffers. The absolute margin over random also narrows at high density, from +51% at medium to +41% at high for DQN.

The metric determined the answer. Reward and crash rate alone supported "PPO generalizes better than DQN". One extra column, how often the policy repeats its most common action, showed that PPO is not generalizing at all. A policy that ignores its input degrades gracefully right up until the environment stops tolerating it, and looks robust on aggregate reward the whole time. This is the most portable finding here: an aggregate-return curve across a shift variable cannot distinguish a robust policy from a policy that is not looking. Some cheap check of whether the policy is a function of the observation belongs alongside it.

Why PPO may have collapsed. This is a plausible explanation rather than something the experiment establishes. The reward is normalized so that simply surviving in the right lane pays roughly 0.73 per step, a

collision both zeroes the step reward and ends the episode, and the speed bonus is worth at most 0.4 of the per-step range. Over a 30 step episode, “never risk anything” is worth about 22 while aggressive driving risks losing the remaining episode. At 50,000 timesteps that is a strong and easily found local optimum. This is a property of the reward design at least as much as of PPO: the generalization question cannot be separated from what the reward pays for.

DQN’s variance is the result, not noise around the result. Three seeds spanning 8.78 to 18.19 at the training density mean the honest summary of DQN at this budget is a distribution wide enough to overlap the control, not a point estimate. Reporting a single seed, in either direction, would have been misleading. Three seeds is the minimum that makes this visible, and it is still too few to characterize the distribution.

What would move this forward, roughly in order of expected value:

1. A longer budget. 50,000 timesteps is short enough that the comparison may be measuring how quickly each algorithm finds a local optimum rather than what it eventually learns.
2. More seeds, given the DQN spread.
3. A reward that does not pay for slow survival, or an evaluation metric independent of the training reward, so that a degenerate policy cannot score well.
4. An observation that scales with density, so that generalization across density is not partly trivialized by a fixed-size observation.
5. Training at more than one density, to distinguish “cannot transfer” from “was never asked to”.

5. Limitations

This section is the important one. The experiment above supports a narrow claim and it is worth being explicit about everything it does not support.

One environment. Every number comes from highway-fast-v0. Nothing here generalizes to other driving scenarios, let alone to other domains. A result about how two algorithms respond to distribution shift in one simulator is a data point, not a finding about the algorithms.

One independent variable, and a narrow range of it. Density was varied over a 3x span (0.5 to 1.5) by scaling vehicle spacing. Everything else was held fixed. Real distribution shift is not one-dimensional.

The traffic model does not change with density. Only the spacing between IDM vehicles changes; their driving behaviour is identical at every level. A real change in traffic conditions involves different driver behaviour, not just different gaps, so this is a weaker form of shift than the framing might suggest.

The observation is partially invariant to the manipulation by construction. The Kinematics observation always reports the 5 nearest vehicles, whatever the density. Raising density changes how close those 5 vehicles are, but not how many the agent sees. Some of the generalization measured here is therefore easier than it would be with a density-sensitive observation, and results with a different observation type could differ.

The action space is not cleanly factored. Under a constant LANE RIGHT policy the ego vehicle decelerates from its 25 m/s starting speed to 20 m/s, whereas a constant IDLE policy holds about 25 m/s. In this version of highway-env the lane-change action

therefore also lowers speed, which means the degenerate policy PPO found is not purely a steering choice. Any account of why that policy is attractive has to include this coupling.

No hyperparameter search. Both algorithms use stable-baselines3 defaults. Those defaults are not equally well suited to every problem, so any gap between DQN and PPO here may reflect the defaults rather than the algorithms. This is the single largest confound in the comparison, and removing it would require a tuning budget per algorithm that this experiment did not have.

A small training budget. 50,000 timesteps is short. Neither agent is near convergence, and results at this budget do not necessarily hold at 500,000. Every statement here is about generalization *at this budget*.

Three seeds. The reported standard deviation over three seeds is a crude uncertainty estimate. No significance testing is performed and none would be meaningful at $n = 3$. Differences smaller than the seed spread should be read as “not resolved by this experiment” rather than as an absence of a difference.

Deterministic evaluation only. Agents are evaluated greedily. A stochastic evaluation policy could behave differently under shift, particularly for PPO, whose training objective is defined over a stochastic policy.

Reward and crash rate are not independent. The environment’s reward already contains the collision penalty and terminates the episode on collision, so a higher crash rate mechanically lowers mean reward. The two metrics are reported together, but they are not two independent pieces of evidence.

No claim about which algorithm is better. The comparison is between two specific implementations, at their default settings, at one budget, in one environment. Read it as a description of what these two runs did, not as a ranking.